



山东大学
SHANDONG UNIVERSITY

网络安全创新创业实践课实验报告

Bitcoin 区块链交易与密码学验证

第 41 组

2026 年 7 月 21 日

小组成员及分工

本实验由小组成员共同完成，各成员主要负责内容如下：

表 1: 小组成员及分工

姓名	学号	班级	主要负责内容
姜良萍（组长）	202300460098	网安 1 班	搭建 Bitcoin Core Testnet4 环境，创建钱包并获取测试币；构造、签名、广播实验交易；获取原始交易与区块数据；实现交易整体结构、输入字段和 UTXO 引用解析；负责最后的格式统一和结果核对
王奕文	202300460056	密码 1 班	实现交易脚本与密码学分析；解析 scriptSig、DER-ECDSA 签名、压缩公钥和 SIGHASH；分析 P2PKH 锁定脚本；计算 TXID、WTXID、手续费和费率；整理交易完整字节解析表
李雨晴	202300460025	网安 2 班	实现完整区块字节解析；解析 80 字节区块头、交易数量和区块内全部交易边界；重新计算区块中各交易的 TXID；构造 Merkle Tree 并验证 Merkle Root；整理区块交易偏移表
曾静雯	202300460045	网安 1 班	实现区块密码学和特殊交易验证；计算区块哈希；解析 nBits 并恢复 Target，验证 PoW；分析 Coinbase 输入、BIP34 高度、区块补贴和手续费；重建 Witness Merkle Tree 并验证 Witness Commitment；撰写实验结论

目录

1	实验任务与总体思路	4
1.1	实验任务	4
1.2	实验对象的选择	5
1.3	总体分析与验证思路	5
2	实验交易与原始数据	6
2.1	实验交易构造	6
2.2	交易广播与区块确认	8
2.3	原始交易与区块数据获取	9
3	交易的字节级解析与验证	10
3.1	交易整体序列化结构	10
3.2	交易输入与 UTXO 引用	11
3.3	解锁脚本与 ECDSA 签名	11
3.4	交易输出与 P2PKH 锁定脚本	12
3.5	TXID、WTXID 与手续费验证	13
4	区块的字节级解析与密码学验证	15
4.1	区块链链接、区块整体结构与 80 字节区块头	15
4.2	区块内交易边界与 Merkle Root	17
4.3	区块哈希与工作量证明	18
4.4	Coinbase 交易分析	19
4.5	Witness Commitment 验证	20
5	实验结论	22
A	实验交易完整字节解析表	23
B	区块内交易偏移表	23

1 实验任务与总体思路

1.1 实验任务

本实验旨在将区块链、区块、交易、输入输出以及 Bitcoin Script 等概念落实到真实的比特币链上数据中。实验首先在 Bitcoin Testnet4 上构造、签名并广播一笔真实交易，待该交易被矿工打包并获得区块确认后，获取其原始交易数据以及所在区块的完整原始数据。

与直接使用区块浏览器或 Bitcoin Core 解码接口查看字段不同，本实验以原始十六进制序列化数据为输入，自行编写 Python 程序维护字节偏移，并按照比特币协议规定的序列化顺序解析交易和区块。在交易层，程序依次解析交易版本、输入数量、前序交易引用、解锁脚本、Sequence、输出数量、金额、锁定脚本和 Locktime；在区块层，程序解析固定长度为 80 字节的区块头、交易数量以及区块中的全部交易。

在完成字段解析的基础上，实验进一步对交易和区块中的关键密码学关系进行独立验证，主要包括：

1. 从原始交易字节中恢复完整的交易结构，并验证所有字节均被准确解析；
2. 分析 P2PKH 交易中解锁脚本与锁定脚本之间的对应关系；
3. 解析 DER 编码的 ECDSA 签名、压缩公钥和签名哈希类型；
4. 从原始交易数据独立计算 TXID 和 WTXID；
5. 解析完整区块及其中的全部交易，重新计算每笔交易的 TXID 和 WTXID；
6. 使用区块内全部交易的 TXID 重建 Merkle Tree，并验证 Merkle Root；
7. 对 80 字节区块头执行双重 SHA-256，重新计算区块哈希；
8. 根据区块头中的 `nBits` 字段恢复完整目标值 Target，并验证工作量证明条件；
9. 分析 Coinbase 交易中的特殊输入、BIP34 区块高度、矿工自定义数据、区块补贴和手续费；
10. 重建 witness Merkle Tree，并验证 Coinbase 交易中的 Witness Commitment。

因此，本实验的重点并非调用现有工具获得格式化结果，而是从底层字节序列出发，理解比特币交易和区块的编码方式，并通过独立计算验证其密码学结构是否成立。

1.2 实验对象的选择

为避免真实资产风险，本实验选择 Bitcoin Testnet4 作为实验网络。Testnet4 与比特币主网采用基本相同的交易、区块和共识数据结构，但测试币不具有实际经济价值，适合进行交易构造、广播和协议分析。

实验交易采用传统的 Legacy P2PKH 类型，并设计为一个输入、一个支付输出和一个找零输出。其结构可表示为

$$1 \text{ Input} \longrightarrow 1 \text{ Payment Output} + 1 \text{ Change Output} + \text{Transaction Fee}.$$

选择该结构主要基于以下原因：

1. 单输入、双输出结构能够清晰体现比特币的 UTXO 模型。交易输入引用一个已经存在的未花费交易输出，该输出被整体消费，剩余金额通过找零输出重新形成新的 UTXO；
2. P2PKH 输入的签名和公钥直接保存在 `scriptSig` 中，便于观察传统解锁脚本的结构；
3. P2PKH 输出采用标准锁定脚本，可以直观分析 `OP_DUP`、`OP_HASH160`、`OP_EQUALVERIFY` 和 `OP_CHECKSIG` 等操作码；
4. Legacy 交易不包含独立的 witness 序列化区域，因此可以直接比较签名前后的交易结构，并验证 `TXID = WTXID`；
5. 虽然实验交易本身采用 Legacy 格式，但其所在区块中包含 SegWit 交易，因此仍可进一步分析区块级的 Witness Commitment。

实验中使用的输入金额为 283468 sat，交易创建一个 100000 sat 的支付输出和一个 182468 sat 的找零输出，交易手续费为 1000 sat，满足

$$283468 = 100000 + 182468 + 1000.$$

该等式同时体现了比特币交易中的资金守恒关系：

$$\sum \text{Input Value} = \sum \text{Output Value} + \text{Transaction Fee}.$$

1.3 总体分析与验证思路

本实验将 Bitcoin Core 与自行编写的解析程序结合使用。其中，Bitcoin Core 主要用于连接 Bitcoin Testnet4 网络、创建测试钱包、完成交易签名、广播交易以及获取交易确认后的原始数据。交易字段解析、脚本分析、哈希计算、Merkle Tree 构造以及工作量证明验证，均由自行编写的 Python 程序基于原始字节独立完成。

整体实验流程如图 1 所示。首先获取实验交易的原始序列化数据 `raw_transaction.hex` 和所在区块的原始数据 `raw_block.hex`，随后分别从交易层和区块层进行分析。

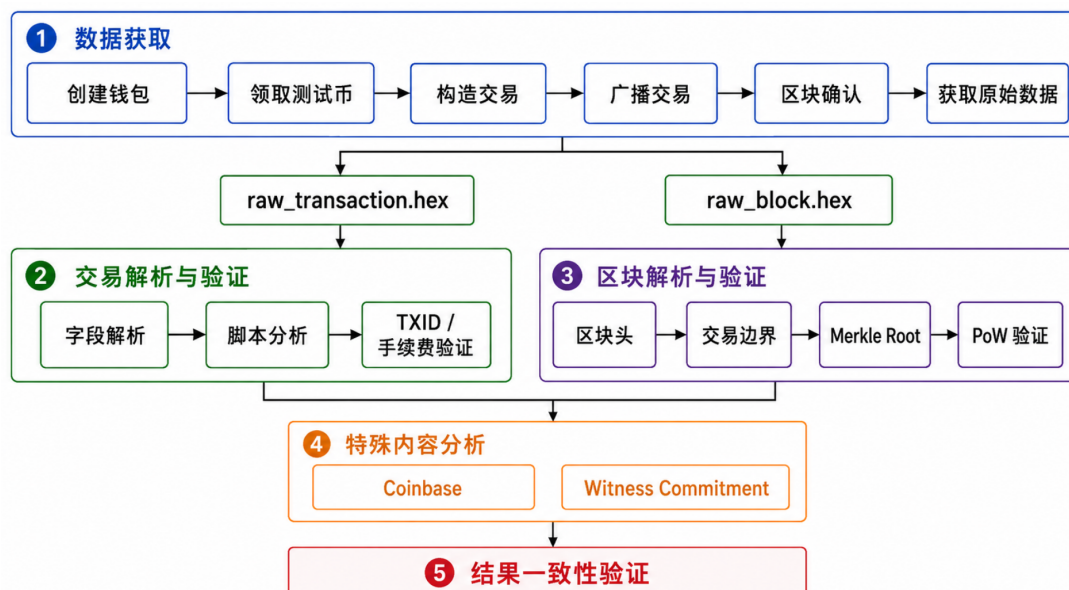


图 1: Bitcoin Testnet4 交易与区块分析总体流程

在交易层，程序按照 Bitcoin 交易序列化格式维护字节偏移，依次解析交易版本、输入输出数量、UTXO 引用、脚本字段以及 Locktime 等信息。对于脚本部分，程序进一步分析 P2PKH 交易中的解锁脚本与锁定脚本，恢复签名、公钥以及地址信息，并通过重新计算 TXID 和 WTXID 验证交易解析结果的正确性。

在区块层，程序首先解析固定长度为 80 字节的区块头，并进一步解析区块中的全部交易。基于交易解析结果，程序重新构造 Merkle Tree，验证计算得到的 Merkle Root 与区块头中的值是否一致；同时，根据区块头中的 `nBits` 字段恢复当前难度目标，并重新计算区块哈希，验证区块是否满足工作量证明条件。

此外，程序进一步分析区块中的 Coinbase 交易和 Witness Commitment，用于验证区块奖励结构以及 SegWit 数据承诺关系。

通过上述流程，本实验实现了从原始交易和区块字节数据到关键密码学结构验证的完整闭环。Bitcoin Core 提供真实链上数据，而所有协议字段解析和密码学验证均由自行编写程序独立完成。

2 实验交易与原始数据

2.1 实验交易构造

本实验选择 Bitcoin Testnet4 作为实验网络，并构造了一笔 Legacy P2PKH 类型的真实交易。为便于后续分析，该交易被设计为一个输入、一个支付输出和一个找零输出

的简单结构。该设计既能够体现比特币 UTXO 模型中“整体花费旧输出并生成新输出”的基本机制，又便于对传统 P2PKH 交易中的解锁脚本与锁定脚本进行分析。

实验交易的整体结构如图 2 所示。交易输入引用一笔已有的 Funding UTXO，其对应的前序交易输出金额为 283468 sat。该输入在本次交易中被整体消费，并生成两个新的输出：其中一个为支付输出，金额为 100000 sat；另一个为找零输出，金额为 182468 sat。此外，交易支付手续费 1000 sat。

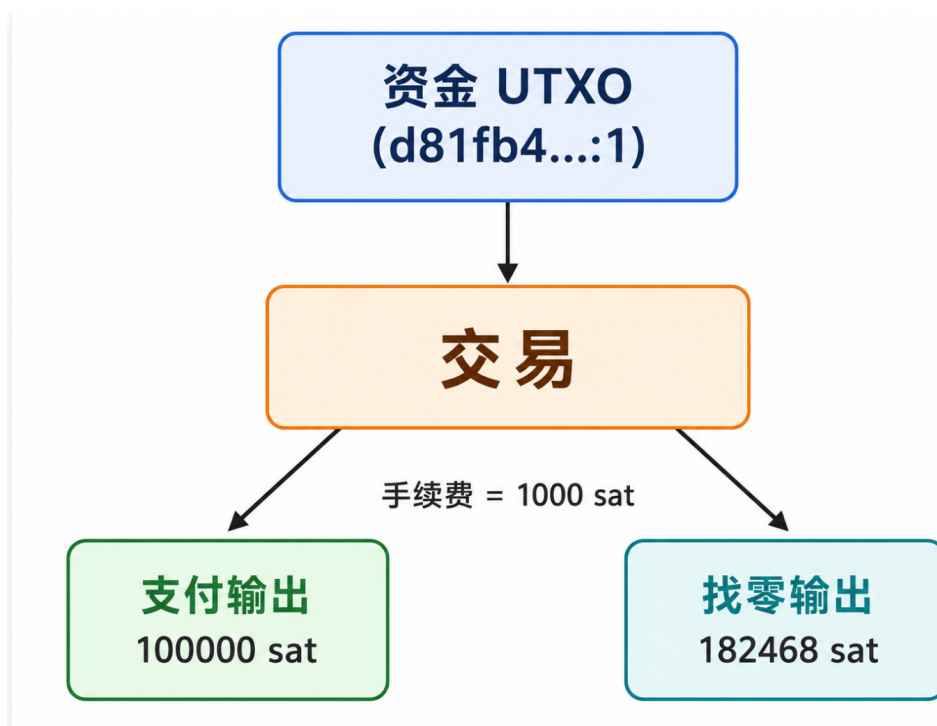


图 2: 实验交易的输入、输出与手续费结构

实验交易的金额分布如表 2 所示。

表 2: 实验交易金额分布

类型	金额 (sat)	说明
Input	283468	消费已有 UTXO
Payment Output	100000	支付目标地址
Change Output	182468	返回找零地址
Fee	1000	交易手续费

该交易满足比特币交易中的资金守恒关系：

$$283468 = 100000 + 182468 + 1000.$$

$$\sum \text{Input Value} = \sum \text{Output Value} + \text{Transaction Fee}.$$

这说明在 UTXO 模型下，输入并不是“账户余额扣减”，而是对旧输出的整体引用与消费，未支付给接收方的剩余金额通过新的找零输出重新回到发送方控制的地址中。

2.2 交易广播与区块确认

实验交易完成构造和签名后，通过本地 Bitcoin Core Testnet4 节点广播到测试网络中。待交易被矿工打包并获得确认后，节点返回了交易的链上确认结果。图 3 展示了交易确认成功后的终端输出，其中包含实验交易的 TXID、WTXID、所在区块高度、区块哈希、区块内索引位置以及原始数据保存路径等关键信息。

```
[m1] 2023-03-03 10:10:10 Confirmations: 6 in mempool 100% cap 21922

=====
EXPERIMENT TRANSACTION CONFIRMED AND CAPTURED
=====

Network      : testnet4
TXID         : 3c6c4e387ea4222795ace0dc46906da2203e7d77d281ee1d8f72685a7d26b4d
f
WTXID        : 3c6c4e387ea4222795ace0dc46906da2203e7d77d281ee1d8f72685a7d26b4d
f
Confirmations : 6
Block height  : 144917
Block hash    : 00000000000000010fd4dcd15b1087d663e3030f6917d3becb43ab22465b2
5
Transaction index : 3
Block transaction(s) : 35
Raw transaction  : 225 bytes
Raw block       : 82923 bytes
Saved transaction : data/raw_transaction.hex
Saved block      : data/raw_block.hex
Metadata updated : data/metadata.json

The transaction is now permanently recorded on Bitcoin Testnet4.
makabaka@LAPTOP-LFHFD31N:/mnt/c/Users/lenovo/Desktop/新建文件夹/大三下/创新创业/day-1/
bitcoin-project$
```

图 3: Bitcoin Testnet4 实验交易确认结果

根据链上确认结果，实验交易的基本信息如表 3 所示。

表 3: 实验交易链上信息

字段	值
Network	Bitcoin Testnet4
TXID	3c6c4e387ea4222795ace0dc46906da 2203e7d77d281ee1d8f72685a7d26b4df
WTXID	3c6c4e387ea4222795ace0dc46906da 2203e7d77d281ee1d8f72685a7d26b4df
Confirmations	6
Block Height	144917
Block Hash	000000000000000010fd4dcdf15b1087 d663e3030f6917d3becb43ab22465b25
Transaction Index	3
Block	
Transaction Count	35

由图 3 和表 3 可见，实验交易已经成功进入区块链，并被包含在高度为 144917 的区块中。由于该交易为 Legacy 交易，不包含独立的 witness 序列化部分，因此其 TXID 与 WTXID 相同。这些链上信息为后续基于原始字节数据的独立解析和密码学验证提供了真实实验对象。

2.3 原始交易与区块数据获取

在交易确认后，实验进一步保存了后续分析所需的原始数据文件，主要包括原始交易、完整区块以及元数据文件。其中文件 `raw_transaction.hex` 保存已签名实验交易的完整十六进制序列化数据，文件 `raw_block.hex` 保存包含该交易的完整区块数据，文件 `metadata.json` 则记录地址、金额、TXID、区块高度等辅助信息。

实验中使用的主要数据文件如表 4 所示。

表 4: 实验原始数据文件

文件名	大小	内容说明
<code>raw_transaction.hex</code>	225 bytes	已签名实验交易的原始序列化数据
<code>raw_block.hex</code>	82923 bytes	包含实验交易的完整区块原始数据
<code>metadata.json</code>	约 3.2 KB	实验辅助元数据

这些原始数据构成了后续实验分析的基础。需要强调的是，后文对交易字段、Bitcoin Script、TXID、Merkle Root、区块哈希以及工作量证明的分析，均直接基于上述原始十

六进制数据完成，而非依赖 Bitcoin Core 或区块浏览器返回的结构化解释结果。也就是说，Bitcoin Core 在本实验中主要负责生成真实链上数据和保存原始数据，而具体的协议解析与密码学验证均由自行编写的 Python 程序独立实现。

3 交易的字节级解析与验证

3.1 交易整体序列化结构

实验交易的原始数据保存在 `data/raw_transaction.hex` 中，共包含 225 字节。本文不直接调用 Bitcoin Core 提供的结构化交易信息，而是从原始十六进制序列化数据出发，根据 Bitcoin 交易格式逐字段解析。

程序维护当前字节偏移量 (offset)，按照交易序列化顺序依次读取 Version、Input Count、交易输入、Output Count、交易输出以及 Locktime 等字段。对于多字节整数，按照 Bitcoin 协议采用小端序解析；对于输入数量、输出数量以及脚本长度等可变字段，则根据 CompactSize 编码进行解析。通过记录每个字段的偏移、长度、原始字节以及解析结果，保证整个交易字节流均被完整覆盖。

该交易属于 Legacy P2PKH 交易，其基本结构为：

$$\text{Transaction} = \text{Version} + \text{Input} + \text{Output} + \text{Locktime}$$

程序解析得到该交易包含：

$$\text{Input Count} = 1, \quad \text{Output Count} = 2.$$

部分关键字段解析结果如表 5 所示。

表 5: 实验交易关键字段解析

偏移	长度	字段	解析结果
0	4	Version	2
4	1	Input Count	1
5	32	Previous TXID	d81fb4...d513
37	4	Previous vout	1
41	1	scriptSig Length	106 bytes
152	1	Output Count	2
221	4	Locktime	0

最终程序验证：

$$\text{Parsed Bytes} = 225,$$

说明所有交易字节均被正确解析。

3.2 交易输入与 UTXO 引用

Bitcoin 采用 UTXO (Unspent Transaction Output) 模型，交易输入并不直接记录转账金额，而是通过引用已有交易输出获得可消费资金。

一个输入可以表示为：

$$\text{Input} = (\text{Previous TXID}, \text{vout}, \text{scriptSig}, \text{Sequence})$$

本实验交易引用的前序输出为：

d81fb4fbc57e7884befd5c8538a27ef7d30ce0bcd57fffb5f16cb16c1f86d513 : 1

该输出金额为：

283468 sat.

因此，本交易并不是直接减少某个账户余额，而是消费一个已有 UTXO，并根据新的输出重新生成后续可花费的 UTXO。这体现了 Bitcoin 交易模型中“输入引用旧状态，输出生成新状态”的特点。

3.3 解锁脚本与 ECDSA 签名

对于 P2PKH 交易，输入中的 scriptSig 用于证明交易发送者拥有对应私钥。程序解析得到：

$$\text{scriptSig Length} = 106 \text{ bytes.}$$

该脚本包含两个数据项：

$$\text{scriptSig} = \langle \text{Signature} + \text{SIGHASH} \rangle + \langle \text{Public Key} \rangle.$$

其中签名部分采用 DER 编码，进一步解析为 ECDSA 签名参数：

$$\text{Signature} = (r, s).$$

程序得到：

$$r = \text{Odd7541e54331f96} \dots$$

$s = 35da01e72573cd63\dots$

同时验证签名满足 Low-S 规范。

需要注意的是，DER 签名结束后的最后一个字节并不属于 DER 结构，而表示签名覆盖范围。本实验中的：

0x01

表示：

SIGHASH_ALL.

该类型表示签名覆盖交易中的全部输出。

程序同时提取压缩公钥，并计算：

$\text{HASH160}(P) = \text{RIPEMD160}(\text{SHA256}(P))$.

计算得到的公钥哈希能够恢复对应 Testnet4 P2PKH 地址，验证该公钥确实对应被消费的旧输出。

图 4 展示了程序对输入脚本和签名信息的解析结果。

```
Input 0
Previous outpoint : d81fb4fbc57e7884befd5c8538a27ef7d30ce0bcd57fffb5f16cb16c1f86d513:1
scriptSig length  : 106 bytes
Sequence          : 4294967294 (0xfffffffffe)
Sequence bits     : 11111111 11111111 11111111 11111110
Witness items     : 0
P2PKH scriptSig   : recognized
Signature DER bytes : 70
r                 : 0dd7541e54331f96a0243abbff0ca83ff21968fb2f11948e55e005f3c450c5d6
s                 : 35da01e72573cd63d681ace5a48899a35d511090eba348c099a3d708abe2030e
Low-S             : True
Sighash           : 0x01 (SIGHASH_ALL)
Sighash bits      : 00000001
Compressed public key: 028cefd2f140d9583c024053a0960cf57260b248bf753e3e22becb00fc26c4b3c1
HASH160(public key) : 1f8a5bc9d7d26b9d1586a155c2c1ee468ab6e49e
Testnet P2PKH address : miPizKaxWLqjo8RtV6uhGKXgr6T4VBawv7
```

图 4: scriptSig 与 ECDSA 签名解析结果

3.4 交易输出与 P2PKH 锁定脚本

交易输出用于定义新的 UTXO 以及未来花费该 UTXO 所需满足的条件。本实验交易包含两个输出，如表 6 所示。

Output	Amount(sat)	类型
0	100000	Payment Output
1	182468	Change Output

两个输出均采用标准 P2PKH 锁定脚本:

$$76 \text{ a9 } 14 \text{ <PubKeyHash> } 88 \text{ ac}$$

其中:

- OP_DUP 复制公钥;
- OP_HASH160 计算公钥哈希;
- OP_EQUALVERIFY 比较哈希值;
- OP_CHECKSIG 验证签名。

因此, 输入中的解锁脚本与输出中的锁定脚本共同构成完整的 P2PKH 花费条件: 用户必须提供匹配的公钥以及有效签名, 才能消费该输出。

3.5 TXID、WTXID 与手续费验证

完成交易字段和脚本解析后, 程序进一步从原始交易数据独立计算交易标识。对于 Legacy 交易, 由于不存在独立的 witness 数据, 因此:

$$\text{TXID} = \text{WTXID}.$$

交易哈希计算过程为:

$$\text{TXID} = \text{Reverse}(\text{SHA256d}(\text{raw transaction})).$$

程序计算得到:

3c6c4e387ea4222795ace0dc46906da2203e7d77d281ee1d8f72685a7d26b4df

与 Bitcoin Core 返回结果一致。

此外, 程序根据输入输出金额验证手续费:

$$\text{Fee} = 283468 - (100000 + 182468) = 1000 \text{ sat}.$$

结合交易虚拟大小:

$$vsize = 225 \text{ bytes},$$

得到手续费率:

$$\text{FeeRate} = \frac{1000}{225} = 4.444 \text{ sat/vB}.$$

图 5 展示了交易哈希、金额以及完整性验证结果。

```
makabaka@LAPTOP-LFHD31N:/mnt/c/Users/lenovo/Desktop/新建文件夹/大三下/创新创业/day-1/bitcoin-project$ OPENSSL_CONF="$PWD/config/openssl-legacy.cnf"
python3 src/parse_transaction.py --no-fields \
| sed -n '/Output 0/,/FINAL RESULT/p'
Output 0
  Amount      : 100000 sat (0.00100000 BTC)
  Script type  : p2pkh
  scriptPubKey : 76a914b10e2fe2a96241e39192b49cb05228c67bd1ec9c88ac
  ASM         : OP_DUP OP_HASH160 b10e2fe2a96241e39192b49cb05228c67bd1ec9c OP_EQUALVERIFY OP_CHECKSIG
  Address      : mwf8tjyGoXsB5vPqUxGZZD9J5pfDybkuQ5

Output 1
  Amount      : 182468 sat (0.00182468 BTC)
  Script type  : p2pkh
  scriptPubKey : 76a914b337b3c612340ab37f13cfaca335cb13c77e7d8e88ac
  ASM         : OP_DUP OP_HASH160 b337b3c612340ab37f13cfaca335cb13c77e7d8e OP_EQUALVERIFY OP_CHECKSIG
  Address      : mwrZyeiyid1QPAYezMmMeWNEgqvkhxTG9T

Total output value : 282468 sat
Known input value  : 283468 sat
Transaction fee     : 1000 sat
Fee rate           : 4.444 sat/vB

=====
PROJECT CROSS-CHECKS
=====
[PASS] TXID matches metadata: actual='3c6c4e387ea4222795ace0dc46906da2203e7d77d281ee1d8f72685a7d26b4df', expected='3c6c4e387ea4222795ace0dc46906da2203e7d77d281ee1d8f72685a7d26b4df'
[PASS] WTXID matches metadata: actual='3c6c4e387ea4222795ace0dc46906da2203e7d77d281ee1d8f72685a7d26b4df', expected='3c6c4e387ea4222795ace0dc46906da2203e7d77d281ee1d8f72685a7d26b4df'
[PASS] Serialized size matches metadata: actual=225, expected=225
[PASS] Input previous TXID: actual='d81fb4fbc57e7884befd5c8538a27ef7d30ce0bcd57fffb5f16cb16c1f86d513', expected='d81fb4fbc57e7884befd5c8538a27ef7d30ce0bcd57fffb5f16cb16c1f86d513'
[PASS] Input previous vout: actual=1, expected=1
[PASS] Output 0 amount: actual=100000, expected=100000
[PASS] Output 1 amount: actual=182468, expected=182468
[PASS] Output 0 address: actual='mwf8tjyGoXsB5vPqUxGZZD9J5pfDybkuQ5', expected='mwf8tjyGoXsB5vPqUxGZZD9J5pfDybkuQ5'
[PASS] Output 1 address: actual='mwrZyeiyid1QPAYezMmMeWNEgqvkhxTG9T', expected='mwrZyeiyid1QPAYezMmMeWNEgqvkhxTG9T'
[PASS] Fee from input minus outputs: actual=1000 sat, expected=1000 sat
[PASS] scriptSig public key recreates funding address: actual='miPizKaxWlqjo8RtV6uhGKXgr6T4VBawv7', expected='miPizKaxWlqjo8RtV6uhGKXgr6T4VBawv7'
[PASS] scriptSig compressed public key: actual='028cefd2f140d9583c024053a0960cf57260b248bf753e3e22becb00fc26c4b3c1', expected='028cefd2f140d9583c024053a0960cf57260b248bf753e3e22becb00fc26c4b3c1'

=====
FINAL RESULT
makabaka@LAPTOP-LFHD31N:/mnt/c/Users/lenovo/Desktop/新建文件夹/大三下/创新创业/day-1/bitcoin-project$
```

图 5: TXID、手续费与交易完整性验证结果

程序最终验证:

- TXID 与链上记录一致;
- WTXID 与链上记录一致;
- 输入输出金额关系正确;
- 手续费计算正确;
- 原始交易字节被完整解析。

4 区块的字节级解析与密码学验证

4.1 区块链链接、区块整体结构与 80 字节区块头

在完成实验交易解析后，本文进一步对该交易所在的完整区块进行分析。原始区块数据保存在 `data/raw_block.hex` 中，共包含 82923 字节。与交易解析过程类似，程序直接读取原始区块序列化数据，根据 Bitcoin 区块格式逐字段解析，而不是依赖区块浏览器提供的结构化信息。

需要注意的是，本文获取的原始区块数据来自 Bitcoin Core 的 `getblock <blockhash> 0` 接口，因此数据从 80 字节区块头开始，不包含网络通信中的 `magic` 和长度字段。

一个完整 Bitcoin 区块可以表示为：

$$Block = Block\ Header + Transaction\ Count + Transactions$$

其中，Block Header 长度固定为 80 字节，随后通过 CompactSize 编码记录区块中的交易数量，并依次存储全部交易。程序解析得到的区块整体结构如图 6 所示。



图 6: Bitcoin 区块整体结构与 80 字节区块头

80 字节区块头由以下字段组成：

表 7: Bitcoin 区块头字段结构

偏移	长度	字段	功能
0	4	Version	区块版本信息
4	32	Previous Block Hash	连接前一区块
36	32	Merkle Root	承诺交易集合
68	4	Timestamp	区块时间戳
72	4	nBits	当前难度目标
76	4	Nonce	挖矿随机数

其中，Previous Block Hash 是区块链结构形成的关键。当前区块保存前一区块的哈希值：

$$Block_i.previous_hash = Hash(Block_{i-1})$$

因此，若历史区块中的任意交易或区块头发生变化，其哈希值也会改变，导致后续区块保存的哈希链接失效。这种链式结构保证了区块历史数据的不可篡改性。

4.2 区块内交易边界与 Merkle Root

区块头之后首先存储交易数量字段。程序解析得到：

$$Transaction\ Count = 35.$$

随后，程序按照交易序列化格式逐笔解析区块中的交易，通过交易内部字段确定每笔交易的长度和边界。

具体而言，对于每个交易：

$$TX_i = Version + Input + Output + Locktime$$

程序读取交易头部字段，并根据输入、输出数量以及脚本长度确定下一笔交易的位置，最终完成整个区块交易集合解析。

实验交易在该区块中的位置为：

$$Transaction\ Index = 3.$$

同时，程序验证：

$$All\ Bytes\ Consumed = True,$$

说明整个区块字节流均被完整解析，不存在遗漏或多余数据。

Bitcoin 区块头并不会直接保存全部交易，而是保存交易集合的摘要值 Merkle Root。Merkle Tree 首先对每笔交易计算 TXID：

$$H_i = SHA256d(TX_i)$$

随后将相邻节点两两组合计算父节点，递归得到最终根节点：

$$MerkleRoot_{computed}.$$

当某一层节点数量为奇数时，复制最后一个节点后继续计算下一层。本区块中 Merkle Tree 的宽度变化为：

$$35 \rightarrow 18 \rightarrow 9 \rightarrow 5 \rightarrow 3 \rightarrow 2 \rightarrow 1.$$

最终程序计算结果：

$$MerkleRoot_{computed} = MerkleRoot_{header}.$$

说明由区块内交易重新构造出的 Merkle Root 与区块头记录完全一致，验证了交易集合的完整性。

4.3 区块哈希与工作量证明

Bitcoin 区块有效性的核心在于工作量证明 (Proof-of-Work)。需要注意的是，PoW 并不是对整个区块进行哈希，而是仅对固定长度 80 字节的区块头计算双 SHA-256：

$$H = SHA256(SHA256(BlockHeader)).$$

程序重新计算得到区块哈希：

```
0000000000000000010fd4dcdf15b1087d663e3030f6917d3becb43ab22465b25
```

与 Bitcoin Core 返回结果一致。

随后，程序解析区块头中的 `nBits` 字段。实验区块中的：

$$nBits = 0x190228f4.$$

其中，高字节表示目标值的指数 E ，低三字节表示系数 C ：

$$E = 0x19, \quad C = 0x0228f4.$$

因此完整目标值为：

$$Target = C \times 256^{E-3}.$$

程序将区块哈希转换为 256 位整数：

$$H_{int}$$

并验证：

$$H_{int} \leq Target.$$

验证结果为 True，说明该区块哈希小于当前难度目标，满足 Bitcoin Testnet4 的工作量证明要求。

图 7 展示了程序对 Merkle Root 和 PoW 的独立验证结果。

```
FINAL RESULT  
makabaka@LAPTOP-LFHF031N:/mnt/c/Users/lenovo/Desktop/新建文件夹/大三下/创新创业/day-1/bitcoin-project$ OPENSSL_CONF="$PWD/config/openssl-legacy.cnf"  
python3 src/pase_block.py --no-transactions \  
| sed -n '/MERKLE ROOT VERIFICATION/, /COINBASE TRANSACTION/p'  
MERKLE ROOT VERIFICATION  
  
=====
```

Header Merkle root	: 7fd2d432e68ff3cccfdc1a50499f39990d2b5635d0683749ef99e3f960b4596d4
Computed Merkle root	: 7fd2d432e68ff3cccfdc1a50499f39990d2b5635d0683749ef99e3f960b4596d4
Merkle level widths	: 35 -> 18 -> 9 -> 5 -> 3 -> 2 -> 1
Merkle root valid	: True

```
=====
```

BLOCK HASH AND PROOF-OF-WORK

```
=====
```

Computed block hash	: 00000000000000010fd4dcdf15b1087d663e3030f6917d3becb43ab22465b25
Target exponent	: 25 (0x19)
Exponent bits	: 00011001
Target sign bit	: 0
Target coefficient	: 0x0228f4
Coefficient bits	: 0000010 00101000 11110100
Expanded target	: 0000000000000228f400
Block hash integer	: 416580554307617718507229551481213749718797824220107004709
Target integer	: 1355837117392573520211088573595572924770889698738874351616
Target - hash	: 13141790619618117483603656184474359175052091874518767346907
Difficulty	: 1988405166.4596343496566729774788776173387210715194
Hash integer <= target	: True

```
=====
```

COINBASE TRANSACTION

```
makabaka@LAPTOP-LFHF031N:/mnt/c/Users/lenovo/Desktop/新建文件夹/大三下/创新创业/day-1/bitcoin-project$
```

图 7: Merkle Root 与区块工作量证明验证结果

4.4 Coinbase 交易分析

区块中的第一笔交易为 Coinbase 交易，与普通交易具有不同的数据结构。普通交易输入通过：

$(TXID, vout)$

引用已有 UTXO，而 Coinbase 交易用于创建新区块奖励，因此不存在真实的前序交易引用。

程序解析得到 Coinbase 输入:

```
Previous TXID = 0000...0000
```

以及：

Previous vout = 0x f f f f f f f f.

因此，该输入被识别为特殊 Coinbase 输入。

Coinbase 的 `scriptSig` 不用于提供签名，而用于存储矿工附加信息。程序进一步解析其中的数据：

- 第一个 Push 数据对应 BIP34 区块高度；
- 第二个 Push 数据为矿工自定义数据，可作为 extraNonce 候选；
- 第三个 Push 数据包含可读矿工标识。

本区块 Coinbase 中解析得到：

$$Block\ Height = 144917$$

并包含矿工标识：

$$/Mined\ @\ SoloPool.Com/.$$

此外，Coinbase 输出金额为：

$$5000042404\ sat.$$

当前区块补贴为：

$$5000000000\ sat.$$

因此矿工额外获得的手续费部分为：

$$5000042404 - 5000000000 = 42404\ sat.$$

该结果满足：

$$CoinbaseReward = BlockSubsidy + TransactionFees.$$

图 8展示了 Coinbase 交易解析结果。

4.5 Witness Commitment 验证

虽然本实验交易采用 Legacy P2PKH 格式，不包含 witness 数据，但所在区块中包含其他 SegWit 交易，因此区块仍需要通过 Witness Commitment 保证 witness 数据完整性。

普通 Merkle Root 使用交易 TXID：

$$TXID$$

而 SegWit 引入新的 witness 数据后，需要额外使用 WTXID 构造 Witness Merkle Tree：

$$WTXID \rightarrow Witness\ Merkle\ Root.$$

随后，将 Witness Merkle Root 与 Coinbase 中保存的 witness reserved value 拼接，并进行双 SHA-256，得到最终 commitment：

$$Commitment = SHA256d(WitnessRoot || ReservedValue).$$

程序重新计算得到：

$$Commitment_{computed} = Commitment_{header}.$$

验证结果：

$$Witness\ Commitment\ Valid = True.$$

说明区块中的 witness 数据承诺关系正确。

```

COINBASE TRANSACTION
=====
Coinbase TXID      : 9372fe14d33410662ad0517ef3c1de6a2c2f7002cbef0a8b1280bb4074977f37
Coinbase WTXID     : 35bc541d4809ad1a64c92755d578315a3ba03cea2ad6d62f6b89a6ed85d70528
Previous TXID      : 0000000000000000000000000000000000000000000000000000000000000000
Previous vout      : 4294967295 (0xffffffff)
Special Coinbase input : True
Coinbase height    : 144917
Coinbase scriptSig  : 031536020c01410c0c00000000000000162f4d696e6564204020536f6c6f506f6f6c2e436f6d2
  Push 0
    Opcode          : PUSH(3)
    Length           : 3 bytes
    Data             : 153602
    Role             : BIP34 block height
    Script number    : 144917
  Push 1
    Opcode          : PUSH(12)
    Length           : 12 bytes
    Data             : 01410c0c0000000000000000
    Role             : miner-defined binary data / extraNonce candidate
  Push 2
    Opcode          : PUSH(22)
    Length           : 22 bytes
    Data             : 2f4d696e6564204020536f6c6f506f6f6c2e436f6d2f
    Role             : printable miner tag/message
    ASCII            : /Mined @ SoloPool.Com/
Coinbase outputs    : 11
Coinbase output total : 5000042404 sat
Scheduled subsidy   : 5000000000 sat
Claimed fee component : 42404 sat
Experiment tx fee    : 1000 sat
Coinbase witness items : 1

=====
WITNESS COMMITMENT
=====
Commitment present   : True
Commitment output index : 10
Committed hash       : 31dcba7565267533b6bc52ac92bad0d46e3a66b639df13df4532b5be17d44017
Computed hash        : 31dcba7565267533b6bc52ac92bad0d46e3a66b639df13df4532b5be17d44017
Reserved value       : 0000000000000000000000000000000000000000000000000000000000000000
0 ^ 0

```

图 8: Coinbase 交易与 Witness Commitment 验证结果

综上，本文从完整区块原始字节出发，完成了区块结构解析、交易集合恢复、Merkle Root 重计算、区块哈希验证、工作量证明检查、Coinbase 分析以及 Witness Commitment 验证，证明该区块满足 Bitcoin Testnet4 的协议要求。

5 实验结论

本实验围绕 Bitcoin Testnet4 环境下真实交易和区块数据展开分析，从原始序列化字节出发，对 Bitcoin 交易结构、Bitcoin Script、区块结构以及相关密码学验证过程进行了完整实现。

在交易层面，实验成功构造并广播了一笔 Legacy P2PKH 交易，并获取了对应的原始交易数据。通过自行编写的解析程序，从十六进制数据中恢复了交易版本、输入、输出、脚本以及 Locktime 等字段，验证了交易字节流的完整性。同时，实验进一步分析了 UTXO 引用关系、P2PKH 解锁脚本和锁定脚本，并解析了 ECDSA 签名、压缩公钥以及 SIGHASH 类型。

在密码学验证方面，程序独立重新计算了交易 TXID 和 WTXID，并验证其与 Bitcoin Core 返回结果一致。对于交易金额关系，实验验证了输入金额、输出金额和手续费之间满足：

$$\sum Input = \sum Output + Fee.$$

在区块层面，实验对包含该交易的完整区块进行了逐字节解析，恢复了 80 字节区块头、交易数量以及全部交易边界。基于解析得到的交易数据，程序重新构造 Merkle Tree 并验证：

$$MerkleRoot_{computed} = MerkleRoot_{header}.$$

同时，实验根据区块头中的 `nBits` 字段恢复目标值，重新计算区块哈希，并验证：

$$H_{int} \leq Target.$$

结果表明，该区块满足 Bitcoin Testnet4 的工作量证明要求。此外，实验进一步分析了 Coinbase 交易中的特殊输入、BIP34 区块高度、矿工附加数据、区块奖励以及 Witness Commitment，验证了区块级数据结构的完整性。

通过本实验，可以进一步理解 Bitcoin 中交易、区块以及密码学机制之间的联系：交易通过输入输出和脚本实现资金转移，多个交易通过 Merkle Tree 形成区块摘要，而区块头通过哈希链和工作量证明保证整个区块链系统的一致性与不可篡改性。

实验过程中仍存在一定不足。例如，本文主要针对 P2PKH 交易进行了深入分析，尚未实现完整 Bitcoin Script 虚拟机，也未覆盖 Taproot、多签等更加复杂的脚本类型。未来可以进一步扩展解析器，使其支持更多交易类型，并实现更加通用的 Bitcoin 协议分析框架。

A 实验交易完整字节解析表

正文中仅展示实验交易的关键字段解析结果。为完整体现从原始十六进制数据到协议字段的转换过程，本附录给出实验交易的完整字节级解析结果。

程序按照 Bitcoin 交易序列化格式维护字节偏移，依次解析 Version、Input、Output 以及 Locktime 等字段。其中，多字节整数按照小端序解析，输入输出数量以及脚本长度按照 CompactSize 编码解析。

表 8: 实验交易完整字节解析表

Offset	Length	Raw Bytes	Field	Parsed Value
0	4	02000000	Version	2
4	1	01	Input Count	1
5	32	d81fb4...d513	Previous TXID	Funding Transaction
37	4	01000000	Previous vout	1
41	1	6a	scriptSig Length	106 bytes
42	106	473044...761cf	scriptSig	DER Signature + Public Key
148	4	feffffff	Sequence	0xffffffff
152	1	02	Output Count	2
153	8	a086010000000000	Output Value	100000 sat
161	25	76a914...88ac	scriptPubKey	P2PKH
186	8	449a020000000000	Output Value	182468 sat
194	25	76a914...88ac	scriptPubKey	P2PKH
219	4	00000000	Locktime	0

通过上述解析，实验交易的全部 225 字节均被映射到对应协议字段，验证了交易序列化结构的正确性。

B 区块内交易偏移表

实验区块共包含 35 笔交易。程序首先解析区块头后的交易数量字段，随后按照交易序列化格式逐笔确定交易边界，并记录每笔交易的起始偏移、序列化大小以及重新计算得到的 TXID。

其中，实验交易位于区块索引 3 的位置，其起始偏移为 1048 字节，序列化大小为 225 字节，与 `raw_transaction.hex` 内容完全一致。

表 9: 区块内交易偏移表

Transaction Index	Offset	Size(bytes)	TXID
0	81	499	9372fe14...7f37
1	580	222	ab2e94bf...1e3e
2	802	246	ca271d03...f31c
3	1048	225	3c6c4e38...b4df
4	1273	41760	ad033083...dc29
5	43033	222	c53f4af6...d100
⋮	⋮	⋮	⋮
34	82700	223	7403210e...b212